



Cancer Workshop 2026 - Module 8: Pathway Enrichment Analysis

Veronique Voisin

October 2026

Contents

Presentation of the dataset	5
Differential Expression	7
0.1 Install and load libraries	7
0.2 Data used for this comparison	8
0.3 Filter low-count genes and TMM-normalize	9
0.4 Design (~0 + group) and named contrast	10
0.5 Dispersion estimation and quasi-likelihood GLM	11
0.6 Calculating ranking score and export full results	12
0.7 Result output	14
0.8 Version of packages and R	14
Defined genelist	17
0.9 Introduction	17
0.10 Goal of the exercise 1	17
0.11 Data	18
0.12 Exercise 1 - prepare the gene lists	18
0.13 Exercise 2 - visualization	29
Ranked gene list	43
0.14 Introduction	43
0.15 Goal of the exercise	43
0.16 Data	43
0.17 Background	43
0.18 Exercise 1 - prepare the gene lists	44

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 Unported License. This means that you are able to copy, share and modify the work, as long as the result is distributed under the same license.

This module will cover how to use differential expression output to generate pathway enrichment results.

Presentation of the dataset

- This module is a continuation of the transcriptomics module 6 and we will use the same dataset. This dataset contains bulk RNA-seq data of a model of breast cancer. The gene counts were deposited to the Gene Expression Omnibus (GEO) database under accession ***GSE222367*** and are publicly available.
- RNA was extracted from MCF7, a breast cancer cell line under different conditions: parental cell line and cell line samples resistant to the drugs palbociclib or abemaciclib.
- Biological replicates were available for each condition.
- RNA-seq library was prepared and sequenced using an Illumina HiSeq 2500 sequencer.
- Sequence reads were mapped to the human reference genome (Gencode GRCh38) using TopHat as aligner.
- The BAM files with mapped reads for each sample were sorted and then used as input for HTSeq 0.11.0 in order to generate read count data at the gene level.
- We retrieved the gene counts and performed differential expression analysis using a negative binomial generalized linear model, followed by pathway enrichment analysis.



Country	USA
Platforms (1)	GPL16791 Illumina HiSeq 2500 (Homo sapiens)
Samples (42)	GSM6921637 MCF7 Parental biol rep 1 GSM6921638 MCF7 Parental biol rep 2 GSM6921639 MCF7 Parental biol rep 3 GSM6921640 MCF7 1.2 Palbo-resistant biol rep 1 GSM6921641 MCF7 1.2 Palbo-resistant biol rep 2 GSM6921642 MCF7 1.2 Palbo-resistant biol rep 3

Figure 1: GEO dataset

Differential Expression

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 Unported License. This means that you are able to copy, share and modify the work, as long as the result is distributed under the same license.

We perform differential expression analysis using a negative binomial generalized linear model using the edgeR package.

0.1 Install and load libraries

```
## CRAN and Bioconductor setup
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")

## List of CRAN packages
cran_packages <- c(
  "tidyverse",
  "ggrepel"
)

# List of Bioconductor packages
bioc_packages <- c(
  "edgeR"
)

# Install CRAN packages if not already installed
for (pkg in cran_packages) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    install.packages(pkg)
  }
}

# Install Bioconductor packages if not already installed
```

```

for (pkg in bioc_packages) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    BiocManager::install(pkg)
  }
}

# Load the libraries

# CRAN packages
library(tidyverse)

## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
## v dplyr      1.2.1      v readr      2.2.0
## v forcats    1.0.1      v stringr   1.6.0
## v ggplot2    4.0.3      v tibble    3.3.1
## v lubridate  1.9.5      v tidyr     1.3.2
## v purrr      1.2.2
## -- Conflicts ----- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts.

library(ggrepel)
# Bioconductor packages
library(edgeR)

## Loading required package: limma

# create the results folder (if it does not exist yet) to save the outputs
dir.create("results", showWarnings = FALSE)

```

Caution

In the Ubuntu (course server) session, the `dir.create()` line above and every `write.csv()` line in this lab use `~/workspace/Module8/results` instead of `results`, so your results are saved in `~/workspace/Module8/results`.

0.2 Data used for this comparison

For this analysis, we are going to compare the Palbo-resistant cell lines (3 biological replicates) versus the parental cell lines (3 biological replicates)

Country	USA
Platforms (1)	GPL16791 Illumina HiSeq 2500 (Homo sapiens)
Samples (42)	GSM6921637 MCF7 Parental biol rep 1 GSM6921638 MCF7 Parental biol rep 2 GSM6921639 MCF7 Parental biol rep 3 GSM6921640 MCF7 1.2 Palbo-resistant biol rep 1 GSM6921641 MCF7 1.2 Palbo-resistant biol rep 2 GSM6921642 MCF7 1.2 Palbo-resistant biol rep 3

Figure 2: GEO dataset

0.2.1 generate a DGE object

```
# 1. Find the count files
files <- list.files(path = "data", pattern = "\\..txt\\.gz$", full.names = TRUE)

# 2. Sample names from the file names (strip the extension)
samples <- sub("_S.*", "", basename(files))

# 3. Read the files into a DGEList
# HTSeq output has no header: column 1 = gene ID, column 2 = count.
# R reads gzipped files directly, so no need to decompress.
dge <- readDGE(files, columns = c(1, 2), header = FALSE, labels = samples)

## Meta tags detected: __no_feature, __ambiguous, __too_low_aQual, __not_aligned, __alignment_not

# 4. Remove HTSeq summary rows (__no_feature, __ambiguous, etc.)
dge <- dge[!grepl("^__", rownames(dge)), , keep.lib.sizes = FALSE]

# 5. (Optional) add group information
dge$samples$group <- factor(
  c("parental", "parental", "parental",
    "palbo_resistant", "palbo_resistant", "palbo_resistant"),
  levels = c("parental", "palbo_resistant") # parental = reference
)
```

0.3 Filter low-count genes and TMM-normalize

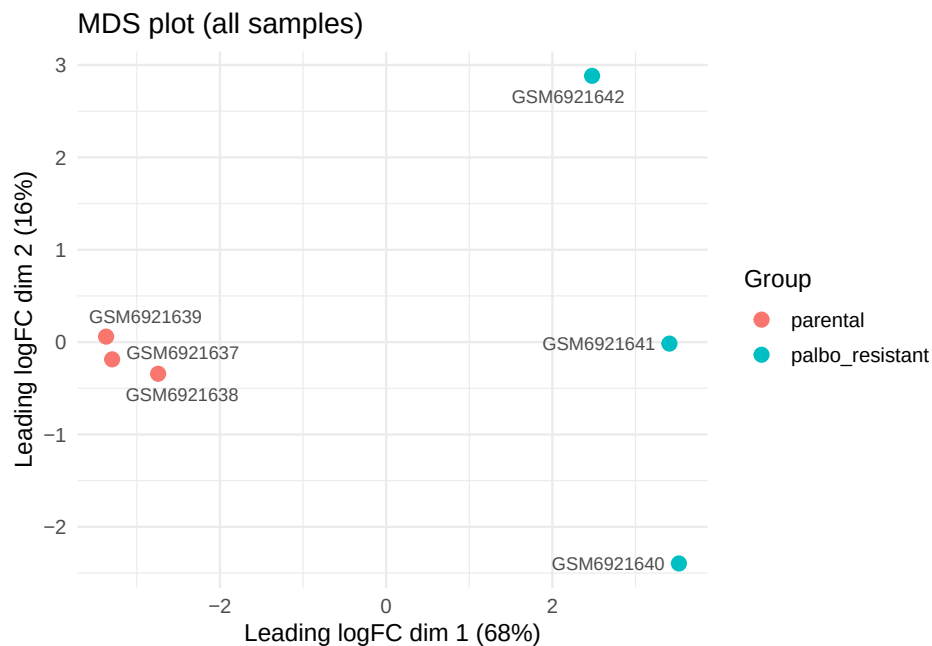
```
keep <- filterByExpr(dge, group = dge$samples$group)
dge <- dge[keep, , keep.lib.sizes = FALSE]
dge <- calcNormFactors(dge, method = "TMM")
cat("Genes retained after filtering:", nrow(dge), "\n")
```

```
## Genes retained after filtering: 15246
```

```
mds <- plotMDS(dge, col = as.numeric(dge$samples$group), main = "MDS (logCPM)")

group = dge$samples$group
var_pct <- round(100 * mds$var.explained[mds$dim.plot])
mds_df <- data.frame(dim1 = mds$x, dim2 = mds$y,
                     sample = colnames(dge), group = group)

ggplot(mds_df, aes(x = dim1, y = dim2, color = group)) +
  geom_point(size = 3) +
  ggrepel::geom_text_repel(aes(label = sample), size = 3, color = "grey30",
                           show.legend = FALSE, max.overlaps = Inf,
                           box.padding = 0.4, segment.color = "grey60") +
  labs(title = "MDS plot (all samples)",
       x = sprintf("Leading logFC dim %d (%d%%)", mds$dim.plot[1], var_pct[1]),
       y = sprintf("Leading logFC dim %d (%d%%)", mds$dim.plot[2], var_pct[2]),
       color = "Group") +
  theme_minimal(base_size = 12) +
  theme(legend.position = "right")
```



0.4 Design ($\sim 0 + \text{group}$) and named contrast

```
design <- model.matrix(~ 0 + group, data = dge$samples)
colnames(design) <- levels(dge$samples$group) # "parental", "palbo_resistant"
```

```

contr <- makeContrasts(
  resistant_vs_parental = palbo_resistant - parental, # + logFC = up in resistant
  levels = design
)

head(design)

```

```

##          parental palbo_resistant
## GSM6921637         1             0
## GSM6921638         1             0
## GSM6921639         1             0
## GSM6921640         0             1
## GSM6921641         0             1
## GSM6921642         0             1

```

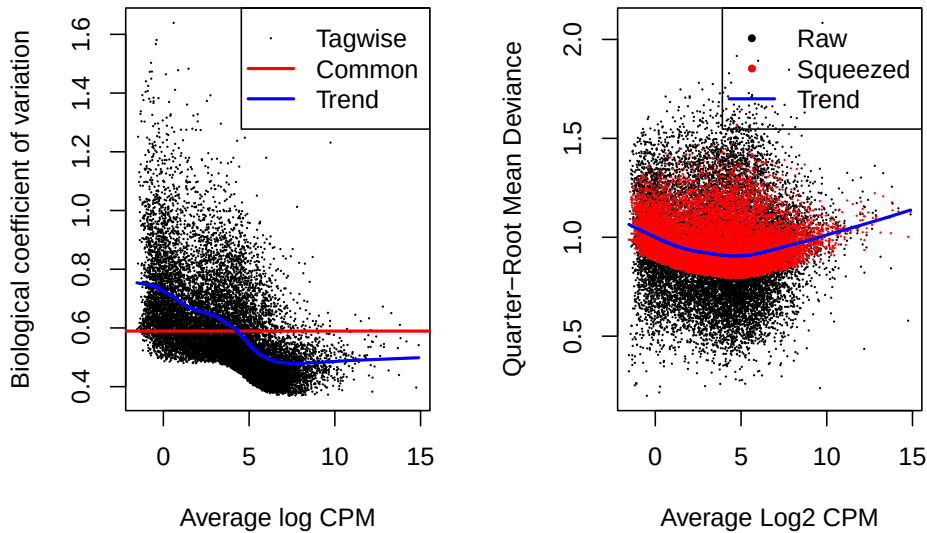
0.5 Dispersion estimation and quasi-likelihood GLM

```

dge <- estimateDisp(dge, design, robust = TRUE)
fit <- glmQLFit(dge, design, robust = TRUE)

par(mfrow = c(1, 2)); plotBCV(dge); plotQLDisp(fit)

```



```

qlf <- glmQLFTest(fit, contrast = contr[, "resistant_vs_parental"])
res <- topTags(qlf, n = Inf, sort.by = "PValue")$table

```

```
res$gene <- rownames(res)
```

0.6 Calculating ranking score and export full results

```
res$score = sign(res$logFC) * -log10(res$PValue)
head(res)
```

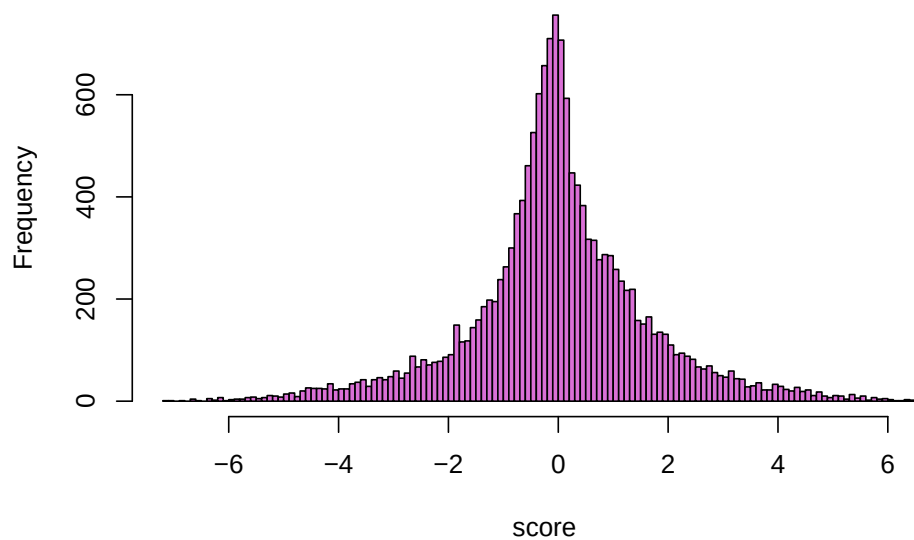
```
##          logFC  logCPM      F      PValue      FDR  gene      score
## FOXA1  -7.733381 7.036614 212.9072 7.319997e-08 0.0003543738 FOXA1 -7.135489
## ESRP1 -12.579669 6.455576 208.4055 8.469988e-08 0.0003543738 ESRP1 -7.072117
## AZGP1 -11.099954 7.258554 184.8560 1.464633e-07 0.0003543738 AZGP1 -6.834271
## EPN3   -11.064332 6.286707 169.9951 2.143977e-07 0.0003543738 EPN3  -6.668780
## AP1M2 -12.059269 6.763487 169.5441 2.169978e-07 0.0003543738 AP1M2 -6.663545
## CAV1   10.118680 6.613522 165.7532 2.404307e-07 0.0003543738 CAV1   6.619010
```

```
##visualization
```

```
### histogram of the ranking score
```

```
hist(res$score, breaks=100, col="orchid", xlab = "score", main = "Histogram of score")
```

Histogram of score



```
### volcano plot
```

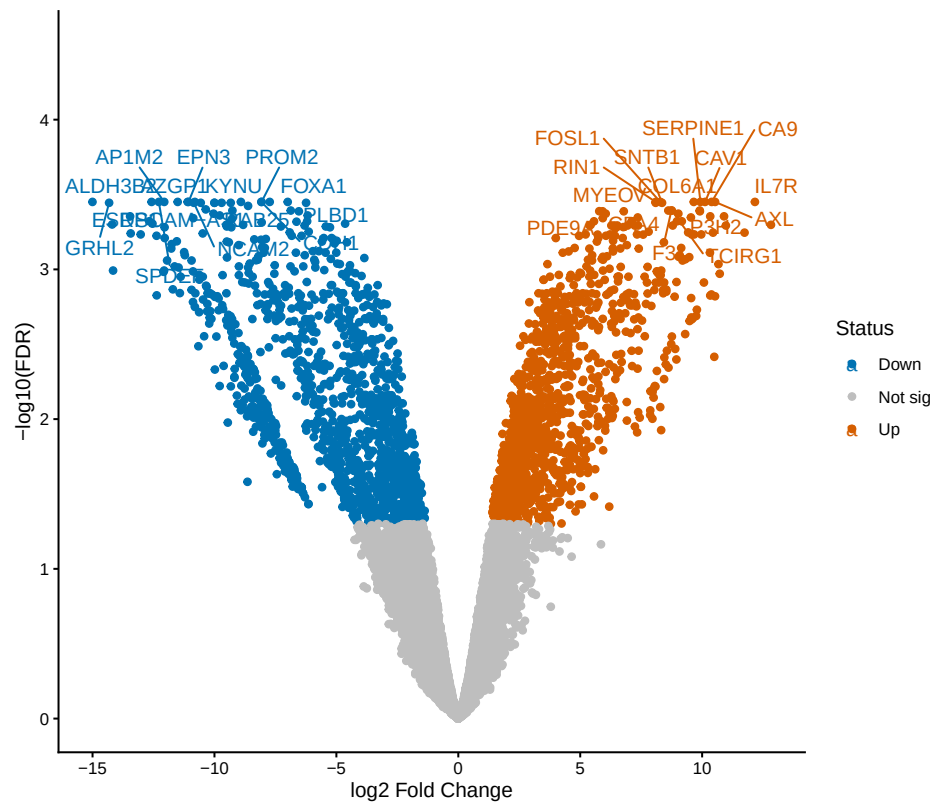
```
# Step 1: add -log10(FDR), and label each gene Up / Down / Not sig
res <- res %>%
```

0.6. CALCULATING RANKING SCORE AND EXPORT FULL RESULTS13

```
mutate(
  negLogFDR = -log10(FDR),
  status = case_when(
    FDR < 0.05 & logFC > 0 ~ "Up",
    FDR < 0.05 & logFC < 0 ~ "Down",
    TRUE ~ "Not sig"
  )
)

# Step 2: pick the top 15 most significant genes on each side, to label
top_up <- res %>% filter(status == "Up") %>% arrange(FDR) %>% slice_head(n = 15)
top_down <- res %>% filter(status == "Down") %>% arrange(FDR) %>% slice_head(n = 15)
top_genes <- bind_rows(top_up, top_down)

# Step 3: plot
ggplot(res, aes(x = logFC, y = negLogFDR, color = status)) +
  geom_point() +
  scale_color_manual(values = c("Up" = "#D55E00", "Down" = "#0072B2", "Not sig" = "grey")) +
  ggrepel::geom_text_repel(data = top_genes, aes(label = gene), max.overlaps = Inf) +
  labs(x = "log2 Fold Change", y = "-log10(FDR)", color = "Status") +
  coord_cartesian(ylim = c(0, 4.5)) +
  theme_classic()
```



0.7 Result output

```
print(summary(decideTests(qlf, adjust.method = "BH", p.value = 0.05)))
```

```
##          -1*parental 1*palbo_resistant
## Down                                1325
## NotSig                             12579
## Up                                  1342
```

```
write.csv(res, "results/edgeR_resistant_vs_parental_all_genes.csv", row.names = FALSE)
```

0.8 Version of packages and R

```
sessionInfo()
```

```
## R version 4.3.2 (2023-10-31)
## Platform: aarch64-apple-darwin20 (64-bit)
## Running under: macOS 15.5
```

```
##
## Matrix products: default
## BLAS: /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/lib/libRblas.0.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.3-arm64/Resources/lib/libRlapack.dylib; LA
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: America/Toronto
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices datasets  utils      methods    base
##
## other attached packages:
## [1] edgeR_4.0.16    limma_3.58.1    ggrepel_0.9.8    lubridate_1.9.5
## [5] forcats_1.0.1   stringr_1.6.0    dplyr_1.2.1      purrr_1.2.2
## [9] readr_2.2.0     tidyr_1.3.2      tibble_3.3.1     ggplot2_4.0.3
## [13] tidyverse_2.0.0
##
## loaded via a namespace (and not attached):
## [1] generics_0.1.4    renv_1.2.4      stringi_1.8.9
## [4] lattice_0.22-7    hms_1.1.4       digest_0.6.39
## [7] magrittr_2.0.5     evaluate_1.0.5   grid_4.3.2
## [10] timechange_0.4.0   RColorBrewer_1.1-3 bookdown_0.48
## [13] fastmap_1.2.0      tinytex_0.61     BiocManager_1.30.27
## [16] scales_1.4.0       cli_3.6.6        rlang_1.3.0
## [19] splines_4.3.2      withr_3.0.3      yaml_2.3.12
## [22] otel_0.2.0         tools_4.3.2      tzdb_0.5.0
## [25] locfit_1.5-9.12    vctrs_0.7.3      R6_2.6.1
## [28] lifecycle_1.0.5    pkgconfig_2.0.3   pillar_1.11.1
## [31] gtable_0.3.6       glue_1.8.1       Rcpp_1.1.2
## [34] statmod_1.5.2      xfun_0.61        tidyselect_1.2.1
## [37] rstudioapi_0.19.0  knitr_1.52       farver_2.1.2
## [40] htmltools_0.5.9    labeling_0.4.3    rmarkdown_2.32
## [43] compiler_4.3.2     S7_0.2.2
```


Defined genelist

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 Unported License. This means that you are able to copy, share and modify the work, as long as the result is distributed under the same license.

Authors: Veronique Voisin, Ruth Isserlin

Presenter: Veronique Voisin

0.9 Introduction

During this practical lab, we will perform pathway enrichment analysis using a defined gene list.

0.10 Goal of the exercise 1

For this exercise, our goal is to run pathway enrichment analysis using g:Profiler and explore the results. g:Profiler performs a gene-set enrichment analysis using a hypergeometric test (Fisher's exact test). The Gene Ontology Biological Process, Reactome and WikiPathways sources are going to be used as pathway databases. We will also upload our custom geneset file.

g:Profiler can be used using the website at <https://biit.cs.ut.ee/gprofiler/gost>. However, for this practical lab, we will run it from the g:Profiler R package.

We will run the query and explore the table of results and visualize the results as bar and dot plots.

One of the greatest features of g:Profiler is that it is updated on a regular basis and most of the previous versions are available online on the gprofiler archive.

The gprofiler2 -g:Profiler R implementation is a wrapper for the web version. You require an internet connection to get enrichment results.

0.11 Data

g:Profiler requires a list of genes. For this analysis, we used genes we found to be differentially expressed in palbociclib-resistant samples compared with parental samples from the breast cancer cell line MCF7 (see chapter 1).

0.12 Exercise 1 - prepare the gene lists

This exercise requires the file “edgeR_resistant_vs_parental_all_genes.csv” located in the data folder.

0.12.1 Step 1 - Install and Load libraries

```
# List of CRAN packages
cran_packages <- c(
  "tidyverse",
  "knitr",
  "kableExtra",
  "glue",
  "patchwork",
  "ggdendro",
  "pheatmap",
  "gprofiler2",
  "GSA"
)
# Install CRAN packages if not already installed
for (pkg in cran_packages) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    install.packages(pkg)
  }
}
# Load the libraries
library(tidyverse)
library(knitr)
library(kableExtra)
library(glue)
library(patchwork)
library(ggdendro)
library(pheatmap)
library(gprofiler2)
library(GSA)

# create the results folder (if it does not exist yet) to save the outputs
dir.create("results", showWarnings = FALSE)
```

Caution

In the Ubuntu (course server) session, the `dir.create()` line above and every `write.csv()` line in this lab use `~/workspace/Module8/results` instead of `results`, so your results are saved in `~/workspace/Module8/results`.

0.12.2 Step 2 - Prepare the define gene lists

```
res = read.csv("data/edgeR_resistant_vs_parental_all_genes.csv")

UPgenes <- res$gene[res$logFC>0 & res$FDR <= 0.001]
DOWNgenes <- res$gene[res$logFC<0 & res$FDR <= 0.001]
```

0.12.3 Step 3 - Run g:Profiler

The next step is to run pathway enrichment analysis using `gost` `g:Profiler` using the `g:Profiler2` R package. For detailed descriptions of all the parameters that can be specified for the `gost g:profiler` function see the R package information - [here](#) and [here](#).

For this query we are specifying -

- `query` - the set of genes of interest, as loaded in from the gene list file (`query_set`).
- `significant` - set to `FALSE` because we want `g:Profiler` to return all the results not just the ones that it deems significant by its predetermined threshold. We will filter the results for significance in a next step.
- `ordered_query` - set to `FALSE` (because we are not taking into account the order of the list)
- `exclude_kea` - set to `TRUE`. We are removing the electronic inferred annotations from the GO database
- `correction_method` - set to `fdr`. by default `g:Profiler` uses `g:Scs`
- `organism` - set to "hsapiens" for homo sapiens. Organism names are constructed by concatenating the first letter of the name and the family name (according to `gprofiler2` documentation)
- `source` - the geneset source databases to use for the analysis. We recommend using GO biological process (GO:BP), WikiPathways (WP) and Reactome (Reac) but there are additional sources you can add (GO molecular function or cellular component(GO:MF, GO:CC), KEGG, transcription factors (TF), microRNA targets (MIRNA), corum complexes (CORUM), Human protein atlas (HPA), Human phenotype ontology (HP))

0.12.3.1 run g:Profiler with the up genes

```
query_set = UPgenes

gprofiler_results_UPgenes <- gost(query = query_set ,
                                   significant=TRUE,
                                   ordered_query = FALSE,
                                   exclude_iea=TRUE,
                                   correction_method = "fdr",
                                   organism = "hsapiens",
                                   source = c("REAC","WP","GO:BP"))
```

0.12.3.2 observe the results

```
#get the gprofiler results table
enrichment_results <- gprofiler_results_UPgenes$result

#display the names of all columns
colnames(enrichment_results)
```

```
## [1] "query"          "significant"      "p_value"
## [4] "term_size"      "query_size"      "intersection_size"
## [7] "precision"      "recall"          "term_id"
## [10] "source"         "term_name"       "effective_domain_size"
## [13] "source_order"   "parents"
```

```
#display the top rows
head(enrichment_results)
```

```
##      query significant      p_value term_size query_size intersection_size
## 1 query_1          TRUE 0.005977645      635         74              13
## 2 query_1          TRUE 0.005977645      509         74              11
## 3 query_1          TRUE 0.005977645       61         74               5
## 4 query_1          TRUE 0.005977645     1013         74              16
## 5 query_1          TRUE 0.005977645      188         74               7
## 6 query_1          TRUE 0.005977645       73         74               5
##      precision      recall      term_id source
## 1 0.17567568 0.02047244 GO:0035295 GO:BP
## 2 0.14864865 0.02161100 GO:0001944 GO:BP
## 3 0.06756757 0.08196721 GO:0050818 GO:BP
## 4 0.21621622 0.01579467 GO:2000026 GO:BP
## 5 0.09459459 0.03723404 GO:0050817 GO:BP
## 6 0.06756757 0.06849315 GO:0032963 GO:BP
##
##                                     term_name effective_domain_size
## 1                                     tube development          16180
## 2                                vasculature development          16180
```

```
## 3 regulation of coagulation 16180
## 4 regulation of multicellular organismal development 16180
## 5 coagulation 16180
## 6 collagen metabolic process 16180
## source_order parents
## 1 8374 G0:0007275, G0:0048856
## 2 542 G0:0048731, G0:0072359
## 3 12418 G0:0050817, G0:0051239
## 4 23538 G0:0007275, G0:0050793, G0:0051239
## 5 12417 G0:0032501
## 6 7434 G0:0008152
```

0.12.3.3 YOUR TURN: run g:Profiler with the down genes (use same parameters as above)

```
##Tip: remove eval=FALSE chunk option to run the code
query_set = XXXX

gprofiler_results_DOWNgenes <- gost(query = XXXX,
  significant = XXXX,
  ordered_query = XXXX,
  exclude_iea = XXXX,
  correction_method = XXXX,
  organism = XXXX,
  source = c("XXXX", "XXXX", "XXXX"))

##observe the results (same as for the UP genes)
```

0.12.4 Clear visualization of the results

The `enrichment_results` dataframe contains different columns. Let's rearrange the table to display the most important information. We will arrange the table so it contains: the database origin, the name of each of the pathway, term size, query size, intersection size and adjusted pvalue. -

- Term size: number of genes in the pathway as it is in the database
- Query size: number of genes in our gene list
- Intersection size: number of genes overlapping between our gene list and the tested pathway
- pvalue: adjusted pvalue (corrected for multiple hypothesis testing using the Benjamini-Hochberg method)

The results are ordered by significance, from the most significant pathway (lowest p-value) to the least significant.

```
enrichment_results <- gprofiler_results_UPgenes$result
enrichment_results$p_value <- formatC(enrichment_results$p_value, format = "e", digits = 2)
```

Table 1: Enrichment Result

source	term_name	term_size	query_size	intersection
GO:BP	tube development	635	74	
GO:BP	vasculature development	509	74	
GO:BP	regulation of coagulation	61	74	
GO:BP	regulation of multicellular organismal development	1013	74	
GO:BP	coagulation	188	74	
GO:BP	collagen metabolic process	73	74	
GO:BP	anatomical structure morphogenesis	1847	74	
GO:BP	cell adhesion mediated by integrin	80	74	
GO:BP	blood vessel morphogenesis	457	74	
GO:BP	tube morphogenesis	562	74	
GO:BP	regulation of multicellular organismal process	2293	74	
GO:BP	blood vessel development	487	74	
GO:BP	positive regulation of blood coagulation	21	74	
GO:BP	positive regulation of hemostasis	21	74	
GO:BP	circulatory system development	745	74	
GO:BP	regulation of blood coagulation	57	74	
GO:BP	regulation of hemostasis	58	74	
GO:BP	wound healing	340	74	
GO:BP	positive regulation of coagulation	23	74	
GO:BP	blood coagulation	182	74	

```
enrichment_results = enrichment_results[ , c("source", "term_name", "term_size", "query")]

kable(head(enrichment_results, n=20), caption = "Enrichment Result") %>%
kable_styling(bootstrap_options = c("striped", "condensed"), font_size = 10)
```

0.12.4.1 YOUR TURN: visualize the results with the down genes (use same parameters as above)

```
##Tip: remove eval=FALSE chunk option to run the code

enrichment_results <- XXXX
enrichment_results$p_value <- formatC(XXXX)

enrichment_results = enrichment_results[ , c("XXXX", "XXXX", "XXXX", "XXXX", "XXXX", "XXXX")]

kable(head(enrichment_results, n=20), caption = "Enrichment Result") %>%
kable_styling(bootstrap_options = c("striped", "condensed"), font_size = 10)
```

Table 2: Enrichment Result

query	significant	p_value	term_size	query_size	intersection_size	precision	recall	term
query_1	TRUE	0.0000077	298	57	12	0.2105263	0.0402685	REAC
query_1	TRUE	0.0008208	89	57	6	0.1052632	0.0674157	REAC
query_1	TRUE	0.0021054	67	57	5	0.0877193	0.0746269	REAC
query_1	TRUE	0.0029160	76	57	5	0.0877193	0.0657895	REAC
query_1	TRUE	0.0033756	44	57	4	0.0701754	0.0909091	REAC
query_1	TRUE	0.0033756	84	57	5	0.0877193	0.0595238	REAC
query_1	TRUE	0.0097759	60	57	4	0.0701754	0.0666667	REAC
query_1	TRUE	0.0256174	140	57	5	0.0877193	0.0357143	REAC
query_1	TRUE	0.0316913	39	57	3	0.0526316	0.0769231	REAC
query_1	TRUE	0.0322168	42	57	3	0.0526316	0.0714286	REAC
query_1	TRUE	0.0322168	516	57	9	0.1578947	0.0174419	REAC
query_1	TRUE	0.0338224	44	57	3	0.0526316	0.0681818	REAC
query_1	TRUE	0.0449470	675	57	10	0.1754386	0.0148148	REAC

0.12.5 Step 4 - Explore top table of results

For this analysis, we chose to use 3 databases: GO:BP, Reactome and Wikipathways. If we look at the top table only, we are only going to see the results of GO:BP but it is interesting to look at the top results for each of the 3 databunique(enrichment_results\$source)

```
enrichment_results <- gprofiler_results_UPgenes$result
##Get the name of the databases
unique(enrichment_results$source)

## [1] "GO:BP" "REAC" "WP"

##Filter the dataframe to retrieve only the Reactome results
enrichment_results_Reac <- enrichment_results %>%
  filter(source == "REAC")

##Now display the top 20 pathways
kable(head(enrichment_results_Reac, n=20), caption = "Enrichment Result") %>%
kable_styling(bootstrap_options = c("striped", "condensed"), font_size = 10)
```

0.12.5.1 YOUR TURN: DISPLAY THE TOP 20 PATHWAYS FOR THE WIKIPATHWAY DATABASE ("WP") and the down genes

```
enrichment_results <- gprofiler_results_DOWNgenes$result
##Get the name of the databases
unique(enrichment_results$source)
```

```
##Filter the dataframe to retrieve only the Reactome results
enrichment_results_WP <- enrichment_results %>%
  XXXX(source == "XXXX")

##Now display the top 20 pathways
kable(head(enrichment_results_WP, n=20), caption = "Enrichment Result") %>%
kable_styling(bootstrap_options = c("striped", "condensed"), font_size = 10)
```

0.12.6 Step 5 - Filter results by geneset size

Restrict the results to just the ones that have at maximum gene-set size of 1000 and a minimum gene-set size of 10 and with an adjusted pvalue of 0.05.-

- maximum gene-set size of 1000: will remove very generic pathway terms that are less informative like “regulation of gene expression”
- minimum gene-set size of 10: will remove small pathways where the pvalue is significant although the overlap size is very small
- p_thres: aim is to retrieve only pathways that are significant under this adjusted pvalue threshold of 0.01 (1% false positive)

```
enrichment_results <- gprofiler_results_UPgenes$result

min_gs_size = 10
max_gs_size = 1000
p_thres = 0.01
# filter by params defined above
# by default we have set the max and min gs size to 250 and 3, respectively.

##the pvalue was stored as a character vector, we have to transform back into a numeric
enrichment_results$p_value <- as.numeric(enrichment_results$p_value)

enrichment_results_mxgssize_1000_min_10_adj_0_01 <-
  subset(enrichment_results,
         term_size >= min_gs_size &
         term_size <= max_gs_size &
         p_value <= p_thres )

#
myrows = nrow(enrichment_results_mxgssize_1000_min_10_adj_0_01)

print(glue("It results in {myrows} selected pathways with maximum gene-set size of {max_gs_size} and minimum gene-set size of {min_gs_size}"))

## It results in 15 selected pathways with maximum gene-set size of 1000 and minimum gene-set size of 10
```

0.12.6.1 YOUR TURN: using the enrichment results for the down genes, use gene-set size filters and FDR filters and display the results

In a second step, try different thresholds of maximum gene-set size and adjusted pvalue threshold: 500 and 250 for * max_gs_size and 0.05 or 0.001 for pvalue-

- Option 1
 - max_gs_size = 500
 - p_thres = 0.001

or

- Option 2
 - max_gs_size = 250
 - p_thres = 0.05

How many pathways do you get for each option (nrow())??

```
enrichment_results <- gprofiler_results_DOWNgenes$result

min_gs_size = XXXX
max_gs_size = XXXX
p_thres = XXXX
# filter by params defined above
# by default we have set the max and min gs size to 250 and 3, respectively.

##the pvalue was stored as a character vector, we have to transform back into a numerical vector
enrichment_results$p_value <- as.numeric(enrichment_results$p_value)

enrichment_results <- XXXXX

#
myrows = nrow(enrichment_results)

print(glue("It results in {myrows} selected pathways with maximum gene-set size of {max_gs_size}")
```

0.12.7 Step 6 - Run g:profiler with your own genesets (example using BaderLab genesets)

With regards to pathway sets there are two options when using g:Profiler -

- Use the genesets that are supplied by g:Profiler as we just did.
- Upload your own genesets.

The most common reasons for supplying your own genesets is the ability to use up to date annotations or in-house annotations that might not be available in the public sphere yet. It can be used to test the enrichment in a particular

pathway of interest that might not be in the general pathway database but that you obtained for example by extracting the data from a published paper. You need to format this pathway in a .gmt file.

In this example, we will upload the Baderlab gene set file that contains multiple database sources. We filtered the gene-set prior to uploading the gene-set file to g:Profiler. The Baderlab gene set file combines different sources of databases and you can find more information at: <https://baderlab.org/GeneSets>

’“Why do we filter the geneset before running gprofiler”’

The g:Profiler interface only allows for filtering genesets by size only after the analysis is complete. After the analysis is complete means the filtering is happening after Multiple hypothesis testing. Filtering prior to the analysis will generate more robust results because we exclude the uninformative large genesets prior to testing changing the sets that multiple hypothesis filtering will get rid of.

We have 1 example of gmt files with different filtering thresholds : * genesets less of equal than 500 genes

Load in the GMT file

This step will take the gmt file that we downloaded locally and upload them into our R environment in the GSA.genesets object called “genesets_baderlab_genesets”. The capture.output() function is only there to redirect the printed messages for a clearer notebook.

The format of the GMT file is described https://software.broadinstitute.org/cancer/software/gsea/wiki/index.php/GMT_file_format and consists of rows with the following

- Name
- Description
- tab delimited list of genes a part of this geneset

Write out the gmt file with genenames

In order to use your own genesets with g:Profiler you need to upload the file to g:Profiler server first. The function will return an ID that you need to specify in the organism parameter of the g:Profiler gost function call. It is done by using the upload_GMT_file() function which is included in the gprofiler2 package. Note: it took about 2 minutes to run this chunk of code.

```
dest_gmt_file_500 ="data/Human_GOBP_AllPathways_filtered_500orLessV2.gmt"

custom_gmt_max500 <- upload_GMT_file(
  gmtfile=dest_gmt_file_500)
```

```
## Your custom annotations ID is gp__et5x_GqJ0_Ibc.
```

```
## You can use this ID as an 'organism' name in all the related enrichment tests again.
```

```
## Just use: gost(my_genes, organism = 'gp__et5x_GqJ0_Ibc')
```

```

UPgenes <- res$gene[res$logFC>0 & res$FDR <= 0.001]
DOWNgenes <- res$gene[res$logFC<0 & res$FDR <= 0.001]

gprofiler_results_custom_max500_UPgenes <- gost(query = UPgenes ,
  significant=TRUE,
  ordered_query = FALSE,
  exclude_iea=TRUE,
  correction_method = "fdr",
  organism = custom_gmt_max500
)

## Detected custom GMT source request
enrichment_results_UPgenes <- gprofiler_results_custom_max500_UPgenes$result

enrichment_results_UPgenes$parents <- sapply(
  enrichment_results_UPgenes$parents,
  function(x) paste(x, collapse = ", ")
)

##Now display the top 20 pathways
kable(head(enrichment_results_UPgenes, n=20), caption = "Enrichment Result") %>%
kable_styling(bootstrap_options = c("striped", "condensed"), font_size = 10)

write.csv( enrichment_results_UPgenes, "results/enrichment_results_gprofiler_UPgenes.csv")

```

0.12.7.1 YOUR TURN: perform the same task with the down genes

```

gprofiler_results_custom_max500_DNgenes <- gost(
  query = XXXX ,
  significant=XXXX,
  ordered_query = XXXX,
  exclude_iea=XXXX,
  correction_method = "XXX",
  organism = XXXX
)

enrichment_results_DNgenes <- gprofiler_results_custom_max500_DNgenes$result

enrichment_results_DNgenes$parents <- sapply(
  enrichment_results_DNgenes$parents,
  function(x) paste(x, collapse = ", ")
)

```

Table 3: Enrichment Result

query	significant	p_value	term_size	query_size	intersection_size	precision	re
query_1	TRUE	0.0000025	308	81	13	0.1604938	0.0422
query_1	TRUE	0.0005125	66	81	6	0.0740741	0.0909
query_1	TRUE	0.0020101	89	81	6	0.0740741	0.0674
query_1	TRUE	0.0043121	62	81	5	0.0617284	0.0806
query_1	TRUE	0.0050573	67	81	5	0.0617284	0.0746
query_1	TRUE	0.0067961	422	81	10	0.1234568	0.0236
query_1	TRUE	0.0086573	40	81	4	0.0493827	0.1000
query_1	TRUE	0.0104444	283	81	8	0.0987654	0.0282
query_1	TRUE	0.0137184	57	81	4	0.0493827	0.0701
query_1	TRUE	0.0137184	98	81	5	0.0617284	0.0510
query_1	TRUE	0.0137184	50	81	4	0.0493827	0.0800
query_1	TRUE	0.0137184	57	81	4	0.0493827	0.0701
query_1	TRUE	0.0137184	21	81	3	0.0370370	0.1428
query_1	TRUE	0.0137184	240	81	7	0.0864198	0.0291
query_1	TRUE	0.0137184	94	81	5	0.0617284	0.0531
query_1	TRUE	0.0137184	56	81	4	0.0493827	0.0714
query_1	TRUE	0.0137184	21	81	3	0.0370370	0.1428
query_1	TRUE	0.0137184	52	81	4	0.0493827	0.0769
query_1	TRUE	0.0139108	58	81	4	0.0493827	0.0689
query_1	TRUE	0.0141423	23	81	3	0.0370370	0.1304

```
kable(head(enrichment_results_DNgenes, n=20), caption = "Enrichment Result") %>%
kable_styling(bootstrap_options = c("striped", "condensed"), font_size = 10)

write.csv( enrichment_results_DNgenes, "results/enrichment_results_gprofiler_DNgenes.csv")
```

0.13 Exercise 2 - visualization

We will use clusterProfiler style visualization. *### Step 1 - Bar plot*

##Calculate a score (small very significant adjusted p value will turn into a large score)

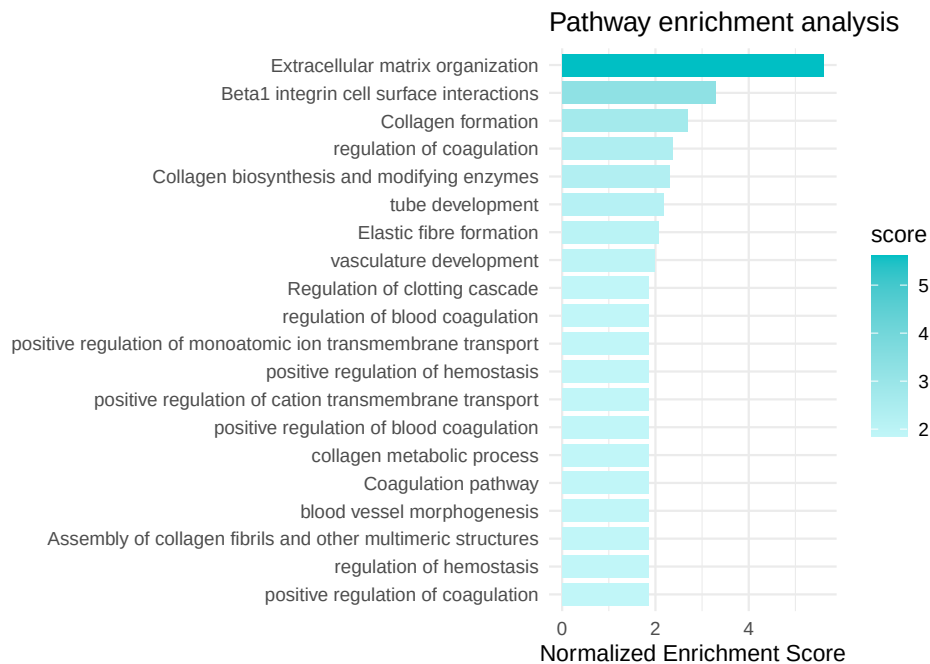
```
enrichment_results_UPgenes <- gprofiler_results_custom_max500_UPgenes$result

enrichment_results_UPgenes$score = -log10(as.numeric(enrichment_results_UPgenes$p_value))

enrichment <- enrichment_results_UPgenes %>%
  as_tibble() %>%
  arrange(desc(score)) %>%
  slice_head(n = 20)

p = ggplot(enrichment, aes(reorder(term_name, score), score)) +
  geom_col(aes(fill = score), width=0.8) +
  scale_fill_gradient(low = "#C1F6F8", high = "#00BFC4")+
  coord_flip() +
  labs(x="", y="Normalized Enrichment Score",title="Pathway enrichment analysis ") +
  theme_minimal()
```

p

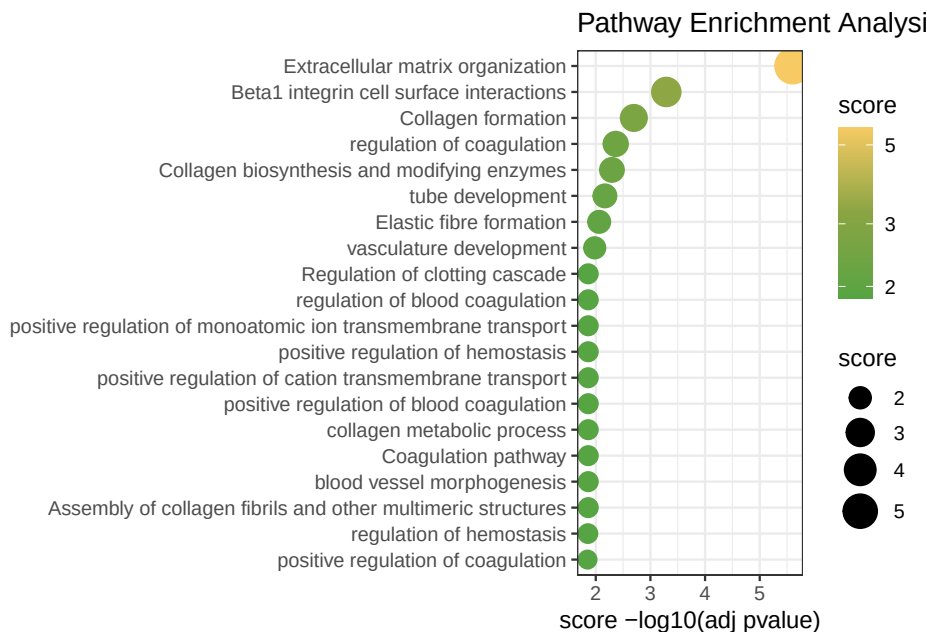


0.13.1 Step 2 - Dot plot with segment

```
enrichmentTop20 <- enrichment %>%
  arrange(desc(score)) %>%
  slice_head(n = 20)

p = ggplot(enrichmentTop20, aes(
  y = fct_reorder(term_name, score),
  x = score,
  color = score,
  size = score
)) +
  geom_point() +
  scale_color_gradientn(
    colours = c("#56A443", "#8AA443", "#f7ca64"),
    trans = "log10",
    guide = guide_colorbar(reverse = FALSE, order = 1)
  ) +
  scale_size_continuous(range = c(4, 8)) +
  theme_bw(base_size = 12) +
  xlab("score -log10(adj pvalue)") +
  ylab(NULL) +
  ggtitle("Pathway Enrichment Analysis") +
```

```
theme(
  axis.text.y = element_text(size = 10)
)
p
```



0.13.2 Step 4 - Hierarchical Cluster

The problem with the earlier visualization, which represents the top 20 pathways, is that it does not account for the overlap between gene sets. As you can observe, many pathways have similar names because they share a substantial number of genes. To provide a more accurate and efficient representation of the results, and to obtain a quicker and clearer overview, it would be more appropriate to aggregate and summarize pathways with substantial gene overlap, thereby reducing redundancy.

Our first approach is to use hierarchical clustering based on a distance matrix that measures the similarity between pathways according to the number of genes they have in common, allowing us to visualize their relationships.

```
enrichment_results_UPgenes <- gprofiler_results_custom_max500_UPgenes$result

enrichment_results_UPgenes$score = -log10(as.numeric(enrichment_results_UPgenes$p_value))

# Top 20 enriched terms
enrichmentTop20 <- enrichment_results_UPgenes %>%
  arrange(desc(score)) %>%
```

```
slice_head(n = 20)
```

0.13.2.1 Get gene list in each pathway

The genes contained in each pathway that overlap with our gene list are unfortunately not included in the enrichment results output. We therefore need to retrieve these genes from the pathway file by extracting the pathways of interest and merging them with our gene list.

addgenes: helper function to add gene list to enrichment result table

```
addgenes = function(gmtfile, enrichmentresult ){

  capt_output <- capture.output(genesets_baderlab_genesets <-
                                GSA.read.gmt(filename = dest_gmt_file_500))

  names(genesets_baderlab_genesets$genesets) <-
    genesets_baderlab_genesets$geneset.names

  subset_genesets <- genesets_baderlab_genesets$genesets[which(genesets_baderlab_genesets

  genes <- lapply(subset_genesets,FUN=function(x){intersect(x,query_set)})

  # For each of the genes collapse to the comma separate text
  genes_collapsed <- unlist(lapply(genes,FUN=function(x){paste(x,collapse = ",")}))

  genes_collapsed_df <- data.frame(
    term_id = names(genes),
    genes = genes_collapsed,stringsAsFactors = FALSE)

  formatted_results <- merge(enrichmentresult,genes_collapsed_df,by.x="term_id" , by.y="

  formatted_results <- formatted_results[order(formatted_results$p_value),]

  return(formatted_results)
}
```

0.13.2.2 Running the function to add genes

```
formatted_results <- addgenes(gmtfile = dest_gmt_file_500 , enrichmentresult= enrichment

head(formatted_results)
```

```

##
## 18
## 2 BETA1 INTEGRIN CELL SURFACE INTERACTIONS%PATHWAY INTERACTION DATABASE NCI-NATURE CURATED DA
## 12
## 42
## 9
## 55
##          query significant      p_value term_size query_size intersection_size
## 18 query_1      TRUE 2.491195e-06      308      81      13
## 2  query_1      TRUE 5.124925e-04      66      81      6
## 12 query_1      TRUE 2.010127e-03      89      81      6
## 42 query_1      TRUE 4.312081e-03      62      81      5
## 9  query_1      TRUE 5.057330e-03      67      81      5
## 55 query_1      TRUE 6.796124e-03      422     81      10
##          precision      recall
## 18 0.16049383 0.04220779 Human_GOBP_AllPathways_filtered_500orLessV2
## 2  0.07407407 0.09090909 Human_GOBP_AllPathways_filtered_500orLessV2
## 12 0.07407407 0.06741573 Human_GOBP_AllPathways_filtered_500orLessV2
## 42 0.06172840 0.08064516 Human_GOBP_AllPathways_filtered_500orLessV2
## 9  0.06172840 0.07462687 Human_GOBP_AllPathways_filtered_500orLessV2
## 55 0.12345679 0.02369668 Human_GOBP_AllPathways_filtered_500orLessV2
##          term_name effective_domain_size
## 18      Extracellular matrix organization      17710
## 2      Beta1 integrin cell surface interactions      17710
## 12      Collagen formation      17710
## 42      regulation of coagulation      17710
## 9      Collagen biosynthesis and modifying enzymes      17710
## 55      tube development      17710
##          source_order parents      score
## 18      3114      NULL 5.603592
## 2      1083      NULL 3.290313
## 12      2582      NULL 2.696776
## 42      14756     NULL 2.365313
## 9      3202      NULL 2.296079
## 55      9957      NULL 2.167739
##
##          genes
## 18 FBLN1,FBN1,SNTB1,ITGA5,TIMP2,P3H2,P3H3,LOX,COL6A2,COL6A1,COL5A1,SERPINE1,BGN
## 2      COL6A1,FBN1,ITGA5,TGFBI,COL5A1,COL6A2
## 12      P3H2,P3H3,LOX,COL6A2,COL6A1,COL5A1
## 42      F2R,CAV1,PROCR,SERPINE1,F3
## 9      P3H2,P3H3,COL6A2,COL6A1,COL5A1
## 55      ERRFI1,TGFBI,LOX,CLMP,SERPINE1,WNK4,ITGA5,RAMP1,CCBE1,CAV1

```

0.13.2.3 Calculating the distance matrix using the jaccard coefficient

- Many pathways share genes, so similar biological processes can appear under different pathway names. To group similar pathways, we look at how many genes they have in common.
- The Jaccard coefficient measures the shared genes relative to all unique genes in the two pathways, from 0 (no overlap) to 1 (identical).
- The overlap coefficient measures the shared genes relative to the smaller pathway and shows when one pathway is mostly contained within another, from 0 (no overlap) to 1 (smaller pathway contained into larger one).

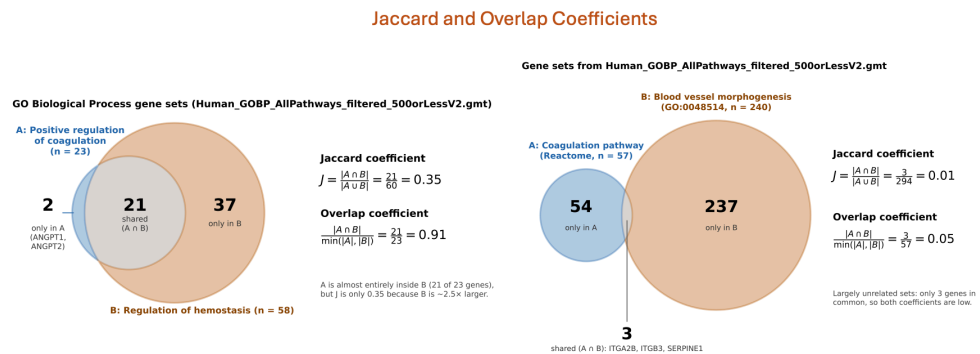


Figure 3: Jaccard and overlap coefficients for two gene sets.

We convert these scores into distances ($1 - \text{score}$) and use them to group similar pathways into broader biological themes.

##Illustration with Jaccard Coefficient

Convert genes to lists

```
gene_lists <- strsplit(formatted_results$genes, ",")
```

Calculate Jaccard similarity matrix

```
n <- length(gene_lists)
```

```
jaccard_mat <- matrix(0, n, n)
```

```
rownames(jaccard_mat) <- formatted_results$term_name
```

```
colnames(jaccard_mat) <- formatted_results$term_name
```

```
for (i in seq_len(n)) {
```

```
  for (j in seq_len(n)) {
```

```
    intersection <- length(intersect(gene_lists[[i]], gene_lists[[j]]))
```

```
    union <- length(union(gene_lists[[i]], gene_lists[[j]]))
```

```
    jaccard_mat[i, j] <- intersection / union
```

```
  }
```

```
}
```

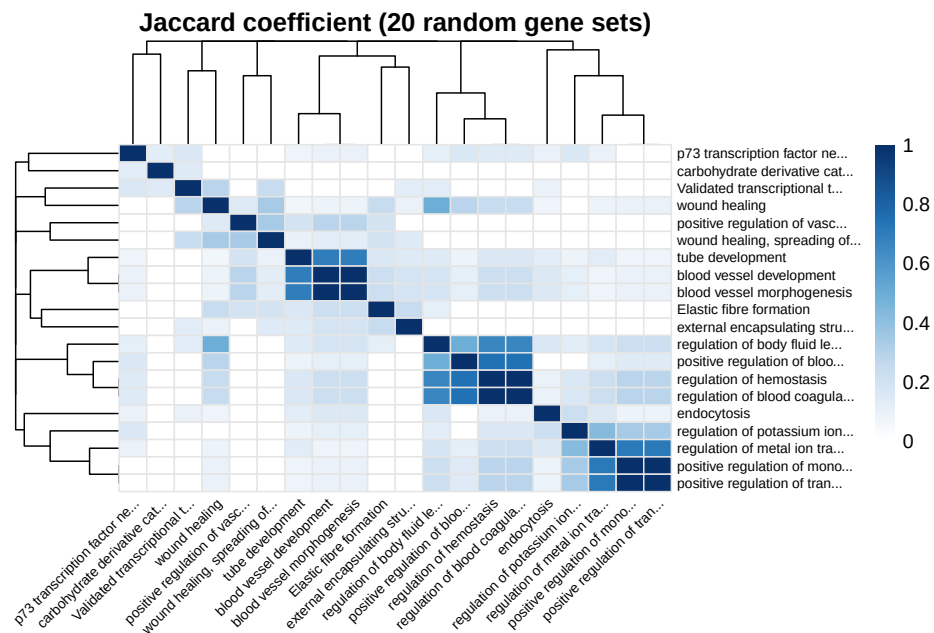
```

labs <- formatted_results$term_name
labs <- ifelse(nchar(labs) > 30, paste0(substr(labs, 1, 27), "..."), labs)
rownames(jaccard_mat) <- colnames(jaccard_mat) <- make.unique(labs)

keep <- sample(seq_len(nrow(jaccard_mat)), min(20, nrow(jaccard_mat)))
jaccard_sub <- jaccard_mat[keep, keep]

# Heatmap
p <- pheatmap(
  jaccard_sub,
  color          = colorRampPalette(c("white", "#C6DBEF", "#6BAED6", "#2171B5", "#08306B"))(100),
  breaks         = seq(0, 1, length.out = 101),
  display_numbers = F,
  #number_format  = "%.2f",
  #number_color   = "black",
  #fontsize_number = 8,
  clustering_distance_rows = as.dist(1 - jaccard_sub),
  clustering_distance_cols = as.dist(1 - jaccard_sub),
  border_color    = "grey90",
  fontsize_row    = 7,
  fontsize_col    = 7,
  angle_col       = 45,
  main            = "Jaccard coefficient (20 random gene sets)"
)

```



##Illustration with Overlap Coefficient

Calculate Overlap coefficient matrix (reuses gene_lists and n from above)
`overlap_mat <- matrix(0, n, n)`

```
for (i in seq_len(n)) {
  for (j in seq_len(n)) {
    intersection <- length(intersect(gene_lists[[i]], gene_lists[[j]]))
    smaller      <- min(length(gene_lists[[i]]), length(gene_lists[[j]]))
    overlap_mat[i, j] <- intersection / smaller
  }
}
```

`rownames(overlap_mat) <- colnames(overlap_mat) <- make.unique(labs)`

Use the same 20 gene sets as the Jaccard heatmap so the two can be compared
`overlap_sub <- overlap_mat[keep, keep]`

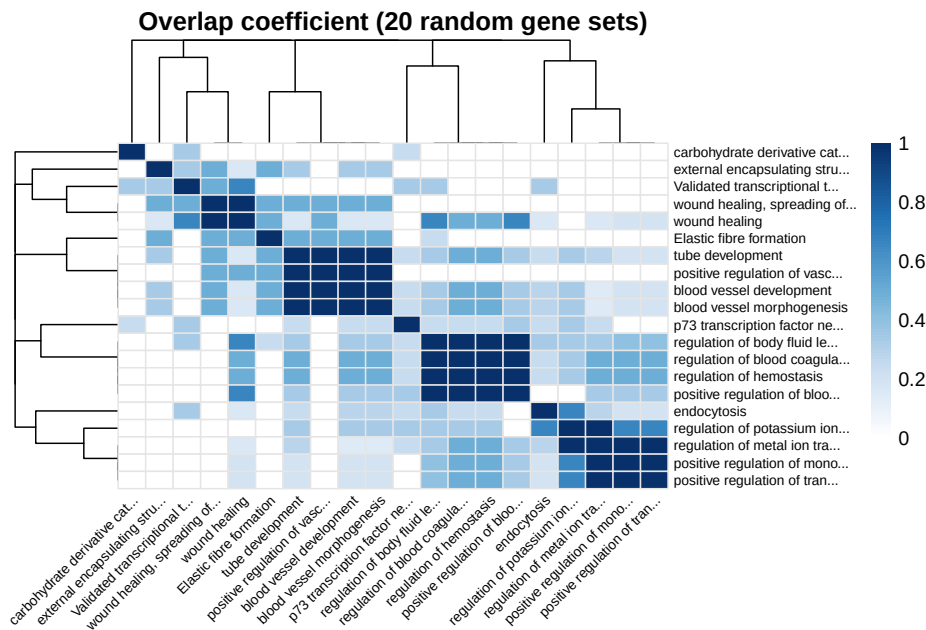
Heatmap

```
p <- pheatmap(
  overlap_sub,
  color          = colorRampPalette(c("white", "#C6DBEF", "#6BAED6", "#2171B5", "#08305B")),
  breaks         = seq(0, 1, length.out = 101),
  display_numbers = F,
  clustering_distance_rows = as.dist(1 - overlap_sub),
```

```

clustering_distance_cols = as.dist(1 - overlap_sub),
border_color             = "grey90",
fontsize_row             = 7,
fontsize_col             = 7,
angle_col                = 45,
main                    = "Overlap coefficient (20 random gene sets)"
)

```



##Hierarchical Clustering

plot_hclust_enrichment: helper function to cluster the top enriched pathways by gene overlap (Jaccard distance) and plot them next to the dendrogram

```

plot_hclust_enrichment = function(enrichment_results, n_top = 20, n_clusters = 8) {

  enrichment_results$score <- -log10(as.numeric(enrichment_results$p_value))

  # Top n_top enriched terms
  enrichmentTop <- enrichment_results %>%
    arrange(desc(score)) %>%
    slice_head(n = n_top)

  # Add the genes of our list found in each pathway
  top_terms <- addgenes(gmtfile = dest_gmt_file_500 , enrichmentresult = enrichmentTop)
  top_terms <- top_terms %>%
    arrange(as.numeric(p_value)) %>%

```

```

slice_head(n = n_top)

# Jaccard similarity matrix
gene_lists <- strsplit(top_terms$genes, ",")
n          <- length(gene_lists)

jaccard_mat <- matrix(0, n, n,
                      dimnames = list(top_terms$term_name, top_terms$term_name))

for (i in seq_len(n)) {
  for (j in seq_len(n)) {
    inter <- length(intersect(gene_lists[[i]], gene_lists[[j]]))
    uni   <- length(union(gene_lists[[i]],      gene_lists[[j]]))
    jaccard_mat[i, j] <- inter / uni
  }
}

# Hierarchical clustering
dist_mat  <- as.dist(1 - jaccard_mat)
hc        <- hclust(dist_mat, method = "average")
term_order <- hc$labels[hc$order]

# The first level is drawn at the bottom of the bar plot, matching the
# first dendrogram leaf (y = 1)
top_terms$term_name <- factor(top_terms$term_name, levels = term_order)
top_terms$neglog10p <- -log10(as.numeric(top_terms$p_value))

# Cut tree into k branches (cannot have more branches than terms)
k          <- min(n_clusters, n)
clusters <- cutree(hc, k = k)

top_terms$cluster <- factor(clusters[as.character(top_terms$term_name)])

# Bar plot (full, coloured by cluster)
base_colours <- c("#1D9E75", "#3A7DC9", "#E07B39", "#9B59B6",
                  "#E74C3C", "#F1C40F", "#1ABC9C", "#E91E63")
# Interpolate extra colours when more than 8 clusters are requested
if (k <= length(base_colours)) {
  cluster_colours <- base_colours
} else {
  cluster_colours <- colorRampPalette(base_colours)(k)
}

p_bar <- ggplot(top_terms, aes(x = neglog10p, y = term_name, fill = cluster)) +
  geom_col(alpha = 0.85, width = 0.7) +

```

```

scale_y_discrete(expand = c(0, 0.5)) +
scale_fill_manual(values = cluster_colours[seq_len(k)],
                  name = "Branch") +
labs(x = expression(-log[10](p-value)), y = NULL) +
theme_bw(base_size = 10) +
theme(
  panel.grid.major.y = element_blank(),
  axis.text.y        = element_text(size = 8),
  legend.position    = "right"
)

# Dendrogram
dend_data <- dendro_data(hc, type = "rectangle")
seg        <- dend_data$segments

p_dend <- ggplot(seg) +
  geom_segment(aes(x = y, y = x, xend = yend, yend = xend),
              colour = "grey40", linewidth = 0.45) +
  scale_x_reverse(expand = expansion(mult = c(0.02, 0))) +
  scale_y_continuous(limits = c(0.5, n + 0.5), expand = c(0, 0)) +
  labs(x = "Jaccard distance") +
  theme_void() +
  theme(
    axis.title.x = element_text(size = 8, colour = "grey50", margin = margin(t = 4)),
    axis.text.x  = element_text(size = 7, colour = "grey60"),
    plot.margin  = margin(0, 4, 0, 0)
  )

# Combined plot
p_combined <- p_dend + p_bar +
  plot_layout(widths = c(3, 1))

# Keep only the top hit per branch
top_per_cluster <- top_terms %>%
  group_by(cluster) %>%
  slice_min(as.numeric(p_value), n = 1, with_ties = FALSE) %>%
  ungroup() %>%
  arrange(as.numeric(p_value)) %>%
  mutate(term_name = factor(as.character(term_name),
                           levels = rev(as.character(term_name))))

# Simple barplot - one representative term per branch
p_simple <- ggplot(top_per_cluster,
                  aes(x = neglog10p, y = term_name, fill = cluster)) +
  geom_col(alpha = 0.88, width = 0.6) +

```

```

geom_text(aes(label = sprintf("p = %.2e", as.numeric(p_value))),
          hjust = -0.1, size = 3, colour = "grey30") +
scale_fill_manual(values = cluster_colours[seq_len(k)],
                  name = "Branch") +
scale_x_continuous(expand = expansion(mult = c(0, 0.25))) +
labs(
  x = expression(-log[10](p-value)),
  y = NULL,
  title = "Top term per branch"
) +
theme_bw(base_size = 11) +
theme(
  panel.grid.major.y = element_blank(),
  axis.text.y = element_text(size = 9),
  legend.position = "bottom",
  plot.title = element_text(size = 11, face = "bold")
)

# n_terms and n_clusters are returned so the figure height can be set
# from them in the chunk options
return(list(combined = p_combined, top_per_branch = p_simple,
           n_terms = n, n_clusters = k))
}

```

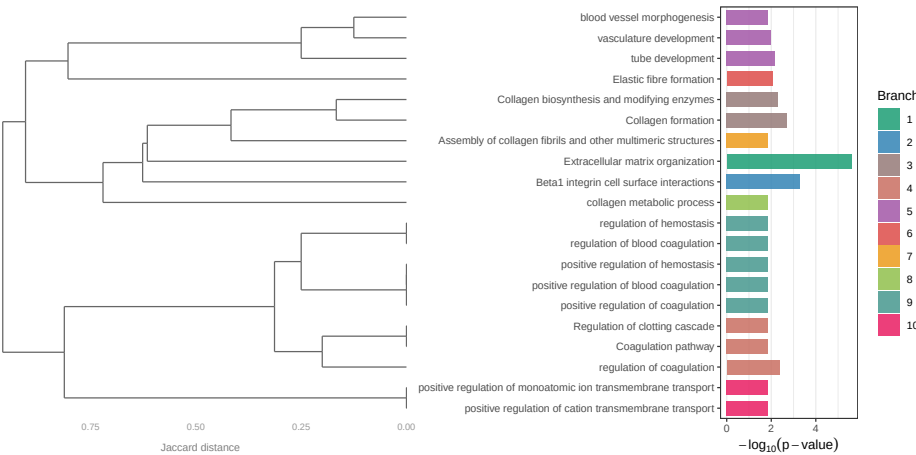
```

enrichment_results_UPgenes <- gprofiler_results_custom_max500_UPgenes$result
hc_plots <- plot_hclust_enrichment(enrichment_results = enrichment_results_UPgenes, n_t

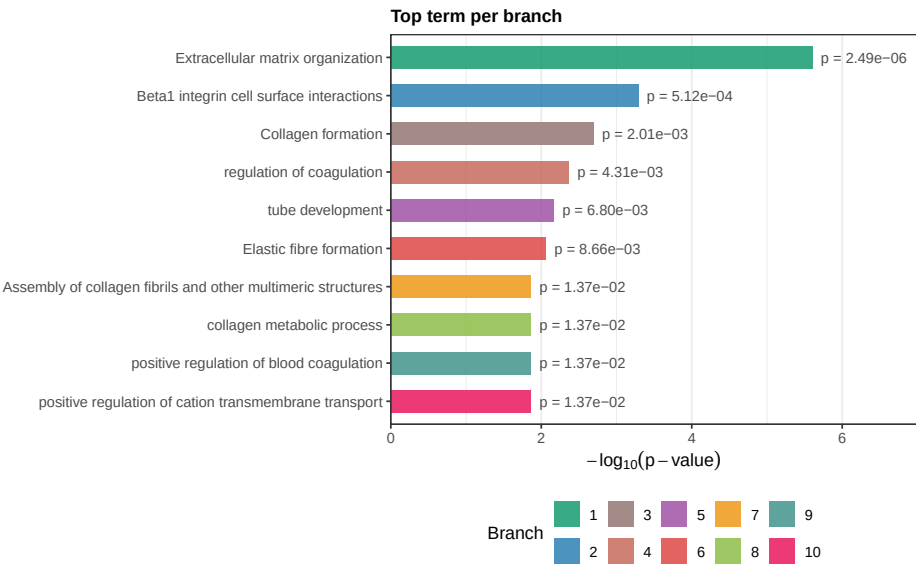
```

The figure height grows with the number of pathways (combined plot) and the number of clusters (top term per branch), so the bars do not overlap.

```
hc_plots$combined
```



`hc_plots$top_per_branch`



0.13.2.5 YOUR TURN Running the function with your defined number of pathways and clusters

Ranked gene list

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 Unported License. This means that you are able to copy, share and modify the work, as long as the result is distributed under the same license.

Authors: Veronique Voisin, Ruth Isserlin

Presenter: Veronique Voisin

0.14 Introduction

This practical lab contains one exercise. It uses fGSEA to perform a gene-set enrichment analysis.

0.15 Goal of the exercise

Learn how to run fGSEA and explore the results.

0.16 Data

We will perform gene-set enrichment analysis using the output of the differential expression analysis comparing the palbo-resistant and the MCF7 parental cell line. We won't use only the significant differentially expressed genes but we will rank all genes from top up-regulated to down-regulated. The non significant genes stays in the rank file.

0.17 Background

The goal of this lab is to:

- Upload the 2 required files into GSEA,
- Adjust relevant parameters,
- Run fGSEA,

- Open and explore the gene-set enrichment results.

The 2 required files are:

1. a rank file (.rnk)
2. a pathway definition file (.gmt).

0.17.0.1 Rank File

To generate a rank file (.rnk), a score $(-\log_{10}(\text{pvalue}) * \text{sign}(\log\text{FC}))$ was calculated from the edgeR differential expression results. A gene that is significantly differentially expressed (i.e associated with a very small pvalue, close to 0) will be assigned a high score. The sign of the logFC indicates if the gene has an expression which is higher in Palbo-resistant ($\log\text{FC} > 0$, the score will have a + sign) or higher in Parental ($\log\text{FC} < 0$, the score will have a - sign). It is used to rank the genes from top up-regulated to top down-regulated (**all genes have to be included**).

0.18 Exercise 1 - prepare the gene lists

This exercise requires the file “edgeR_resistant_vs_parental_all_genes.csv” located in the data folder.

0.18.1 Step 1 - Install and Load libraries

```
# CRAN and Bioconductor setup
if (!requireNamespace("BiocManager", quietly = TRUE))
  install.packages("BiocManager")
# List of CRAN packages
cran_packages <- c(
  "tidyverse",
  "knitr",
  "kableExtra",
  "glue",
  "ggribbles",
  "data.table"
)
# List of Bioconductor packages
bioc_packages <- c(
  "fgsea",
  "clusterProfiler",
  "enrichplot"
)
# Install CRAN packages if not already installed
for (pkg in cran_packages) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
```

```
    install.packages(pkg)
  }
}
# Install Bioconductor packages if not already installed
for (pkg in bioc_packages) {
  if (!requireNamespace(pkg, quietly = TRUE)) {
    BiocManager::install(pkg)
  }
}
# Load the libraries
library(tidyverse)
library(knitr)
library(kableExtra)
library(glue)
library(ggthemes)
library(data.table)

# Bioconductor packages
library(fgsea)
library(clusterProfiler)
library(enrichplot)

# create the results folder (if it does not exist yet) to save the outputs
dir.create("results", showWarnings = FALSE)
```

Caution

In the Ubuntu (course server) session, the `dir.create()` line above and every `write.csv()` line in this lab use `~/workspace/Module8/results` instead of `results`, so your results are saved in `~/workspace/Module8/results`.

0.18.2 Step 2 - Prepare the rank file , upload the gmt file and run fgsea

fGSEA is an R package that runs a fast Gene Set Enrichment Analysis.

In the below command the following options have been specified:

- pathways - list of genesets to use for the calculation
- stats - genes and their associated statistic, sorted (rank file)
- max_size - maximum size for individual gene sets.
- min_size - minimum size for individual gene sets
- gseaParam - GSEA parameter value

Caution

gseaParam - GSEA parameter value. All ranks will be raised to this power when calculating the ES scores. Equivalent to the weight in the original GSEA algorithm.

```
res = read.csv("data/edgeR_resistant_vs_parental_all_genes.csv")

gmtfile = "data/Human_GOBP_AllPathways_noPFOCR_no_GO_iea_September_14_2026_symbol.gmt"
pathways <- fgsea::gmtPathways(gmtfile)

## rank file formatted for gsea
ranks <- setNames(res$score, res$gene)
ranks <- ranks[!is.na(ranks) & !is.na(names(ranks))]
ranks <- sort(ranks, decreasing = TRUE)

set.seed(543) ## to get reproducible results

## running fgsea
fgseaRes <- fgsea(

  pathways = pathways,
  stats = ranks,
  minSize = 10,
  maxSize = 500,
  nperm = 2000
)

## Warning in fgsea(pathways = pathways, stats = ranks, minSize = 10, maxSize =
## 500, : You are trying to run fgseaSimple. It is recommended to use
## fgseaMultilevel. To run fgseaMultilevel, you need to remove the nperm argument
## in the fgsea function call.

## Warning in preparePathwaysAndStats(pathways, stats, minSize, maxSize, gseaParam, :
## The order of those tied genes will be arbitrary, which may produce unexpected results
fgseaRes <- fgseaRes[order(fgseaRes$padj), ]
```

Save the full fgsea results table to the results folder.

```
#the leadingEdge column is a list of genes for each pathway,
#convert it to text (genes separated by a comma) so that it can be written to a csv file
fgseaRes_to_save <- copy(fgseaRes)
fgseaRes_to_save$leadingEdge <- sapply(fgseaRes_to_save$leadingEdge, paste, collapse = ",")
write.csv(fgseaRes_to_save, "results/fgsea_results_resistant_vs_parental.csv", row.names = FALSE)
```

Tip

Set `gseaParam` to 2 if you want to add more weight on the most top up-regulated and top down-regulated. **2** is a more stringent parameter and it will result in less gene-sets significant under FDR <0.05.

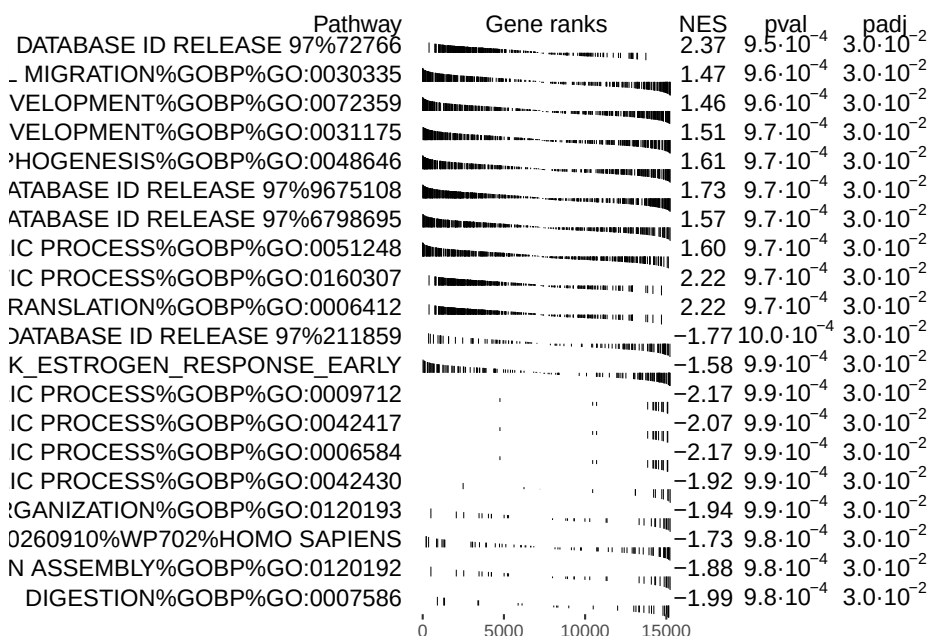
0.18.3 Step 3 - Examine the results

Examining the results

Get the top results and visualize as table

```
topPathwaysUp <- fgseaRes[ES > 0][head(order(pval), n=10), pathway]
topPathwaysDown <- fgseaRes[ES < 0][head(order(pval), n=10), pathway]
topPathways <- c(topPathwaysUp, rev(topPathwaysDown))
plotGseaTable(pathways[topPathways], ranks, fgseaRes, gseaParam=0.5)
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## i Please use `linewidth` instead.
## i The deprecated feature was likely used in the fgsea package.
## Please report the issue at <https://github.com/ctlab/fgsea/issues>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```



When examining the results there are a few things to look for:

Check the number of gene-sets that have been used for the analysis.

Tip

A small number (a few hundred genesets if using baderlab genesets) could indicate an issue with identifier mapping.

Check the number of sets that have FDR less than 0.25 – in order to determine what thresholds to start with when creating the enrichment map. It is not uncommon to see a thousand gene sets pass the threshold of FDR less than 0.25. FDR less than 0.25 is a very lax threshold and for robust data we can set thresholds of FDR less than 0.05 or lower.

Number of pathways with corrected pvalue < 0.05

```
length(which(fgseaRes$padj < 0.05))
```

```
## [1] 325
```

Number of pathways with corrected pvalue < 0.01

```
length(which(fgseaRes$padj < 0.01))
```

```
## [1] 0
```

Number of Palbo-resistant (up -regulated) pathways with corrected pvalue < 0.05

```
length(which(fgseaRes$padj < 0.05 & fgseaRes$ES > 0))
```

```
## [1] 297
```

Number of Parental (down -regulated) pathways with corrected pvalue < 0.05

```
length(which(fgseaRes$padj < 0.05 & fgseaRes$ES < 0))
```

```
## [1] 28
```

0.18.4 Step 4 - Explore the tabular format of the results

0.18.4.1 Palbo-resistant (up-regulated genes)

```
topPathwaysUp <- fgseaRes[ES > 0][head(order(pval), n=5), pathway]
```

```
top_palbo_hits <- fgseaRes[which(fgseaRes$pathway %in% topPathwaysUp),]
```

```
#format the pathway name colum so that it is easier to see the whole table
top_palbo_hits$pathway <- substr(top_palbo_hits$pathway, start = 1, stop=35)
```

```
kable(head(top_palbo_hits), caption = "Top Palbo-resistant pathways") %>%
kable_styling(bootstrap_options = c("striped", "condensed"), font_size = 10)
```

Table 4: Top Palbo-resistant pathways

pathway	pval	padj	ES	NES	nMore
TRANSLATION%REACTOME DATABASE ID RE	0.0009524	0.0297521	0.5492338	2.367385	
POSITIVE REGULATION OF CELL MIGRATI	0.0009615	0.0297521	0.3432083	1.470604	
NEURON PROJECTION DEVELOPMENT%GOBP%	0.0009671	0.0297521	0.3491315	1.507575	
CIRCULATORY SYSTEM DEVELOPMENT%GOBP	0.0009615	0.0297521	0.3364868	1.456819	
ANATOMICAL STRUCTURE FORMATION INVO	0.0009690	0.0297521	0.3734786	1.610248	

Table 5: Top Parental pathways

pathway	pval	padj	ES	NES	nMore
METAPATHWAY BIOTRANSFORMATION PHASE	0.0009843	0.0297521	-0.4964078	-1.728867	
TIGHT JUNCTION ORGANIZATION%GOBP%GO	0.0009872	0.0297521	-0.6351792	-1.936928	
INDOLE-CONTAINING COMPOUND METABOLI	0.0009881	0.0297521	-0.7990183	-1.920113	
DIGESTION%GOBP%GO:0007586	0.0009804	0.0297521	-0.6713524	-1.987560	
TIGHT JUNCTION ASSEMBLY%GOBP%GO:012	0.0009833	0.0297521	-0.6194630	-1.878603	

0.18.4.2 Parental (down-regulated genes)

```
topPathwaysDown <- fgseaRes[ES < 0][head(order(pval), n=5), pathway]

top_parental_hits <- fgseaRes[which(fgseaRes$pathway %in% topPathwaysDown),]

#format the pathway name colum so that it is easier to see the whole table
top_parental_hits$pathway <- substr(top_parental_hits$pathway, start = 1, stop=35)

kable(head(top_parental_hits), caption = "Top Parental pathways") %>%
kable_styling(bootstrap_options = c("striped", "condensed"), font_size = 10)
```

0.18.5 Step 5 - Visualize the top results

Visualize the top results as different sorts of plots, as demonstrated in the first part of the lab.

0.18.5.1 GSEA style Enrichment Plots

In order to create GSEA Enrichment plots we need to use the packages:

- clusterProfiler and
- enrichplot

clusterProfiler runs with its own set of pathways that it parses from GO or KEGG or MSigDB or other sources that are coded into the package. It does not take as input the standard GMT file. If you want to use your own pathway

definitions you need to translate it into a dataframe where for every gene in a given pathway there is a row with the pathway name and the gene.

In order to generate an enrichment plot with `enrichplot` we need the fgsea results in the datastructure returned by `clusterProfiler`. To run fgSEA in this way we need:

- the ranks file, as before
- the pathways as a data frame.

Below we show how to run fgSEA using the `clusterProfiler` package and user supplied pathway definitions.

```
#translate our gmt file (a named list of genesets) into a dataframe
#with one row per gene in each pathway.
TERMS <- data.frame(
  name = rep(names(pathways), lengths(pathways)),
  gene = unlist(pathways, use.names = FALSE)
)

# specify the gene ranks, and make sure it is in descending order.
gsea_scores <- ranks
gsea_scores <- gsea_scores[!is.na(gsea_scores)]
gsea_scores <- sort(gsea_scores, decreasing = TRUE)

# Set a seed for reproducibility
set.seed(123)

#Run fgsea using clusterProfiler, with the same gene-set size limits as above.
selected_gsea <- clusterProfiler::GSEA(
  geneList = gsea_scores,
  TERM2GENE = TERMS,
  minGSSize = 10,
  maxGSSize = 500,
  pvalueCutoff = 1,
  verbose = FALSE,
  pAdjustMethod = "fdr",
  scoreType = "std",
  eps = 0
)

## Warning in preparePathwaysAndStats(pathways, stats, minSize, maxSize, gseaParam, : 
## The order of those tied genes will be arbitrary, which may produce unexpected results

Print out a few of the top enrichment plots
all_figures <- list()

#generate GSEA plots for the top Parental (down-regulated) genesets that we have already
```

```

rows_to_use <- which(selected_gsea@result$ID %in% topPathwaysDown)

for(i in seq_along(rows_to_use)){
  row_number = rows_to_use[i]

  #format the title for the enrichment plot
  pw_des <- selected_gsea@result$Description[row_number]
  pw_pval <- selected_gsea@result$pvalue[row_number]
  pw_pval <- format(pw_pval, scientific = F, digits = 3)
  pw_qval <- selected_gsea@result$p.adjust[row_number]
  pw_qval <- format(pw_qval, scientific = F, digits = 3)
  pw_nes <- selected_gsea@result$NES[row_number]
  pw_nes <- format(pw_nes, scientific = F, digits = 2)
  pw_es <- selected_gsea@result$enrichmentScore[row_number]
  pw_es <- format(pw_es, scientific = F, digits = 2)
  mytitle <- glue::glue('{pw_des}\n(pval: {pw_pval}, qval: {pw_qval}, NES: {pw_nes}, ES: {pw_es})')

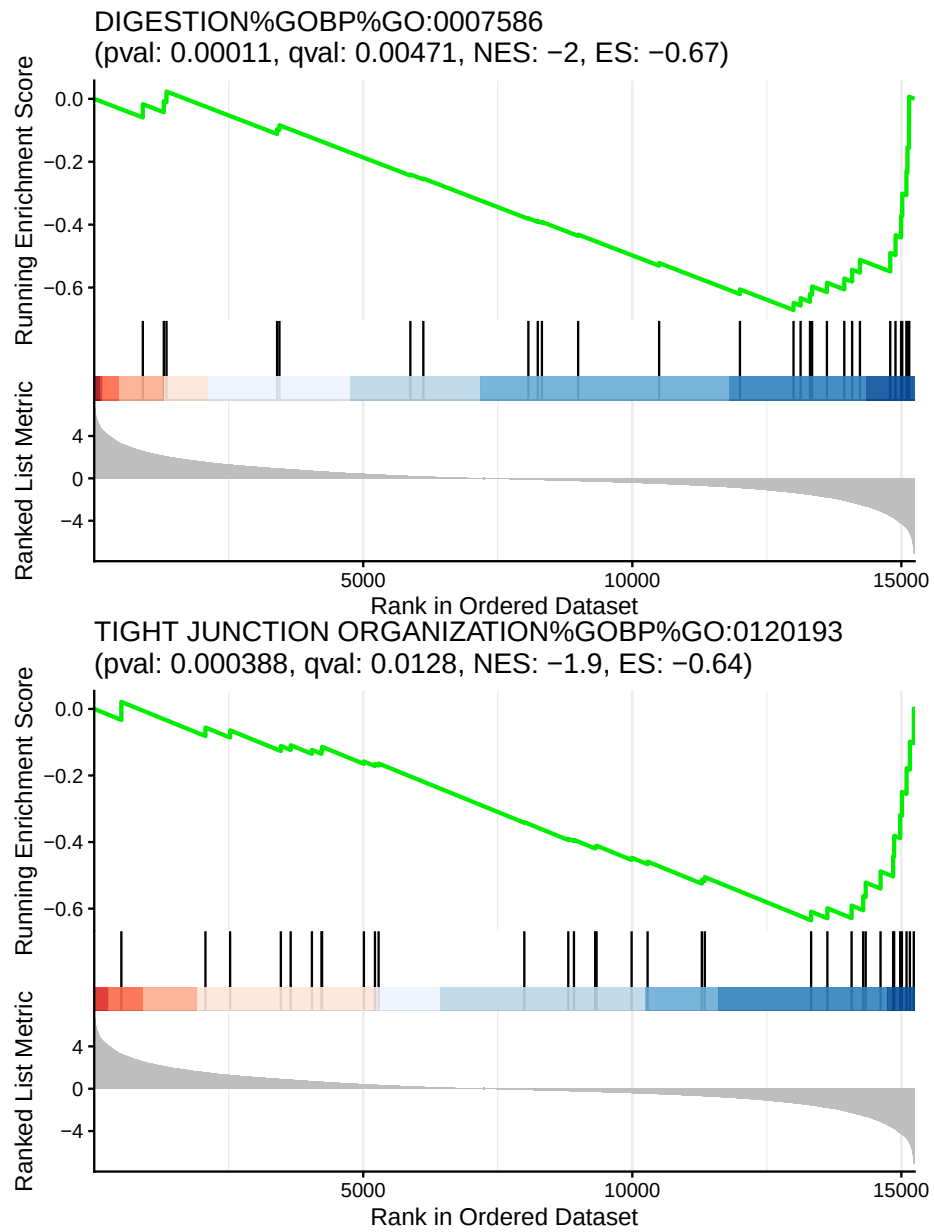
  figure <- enrichplot::gseaplot2(
    selected_gsea,
    geneSetID = selected_gsea@result$ID[row_number],
    title = mytitle,
    color = "green2",
    base_size = 12,
    rel_heights = c(1.5, 0.5, 1),
    subplots = 1:3,
    pvalue_table = F,
    ES_geom = "line"
  )

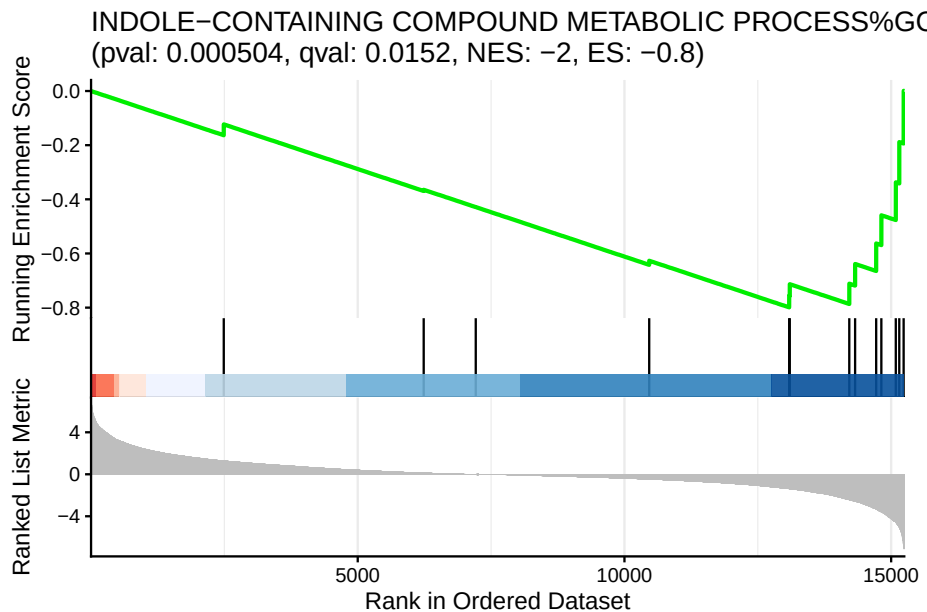
  all_figures[[i]] <- figure
}

## Warning: `aes()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`
## i The deprecated feature was likely used in the enrichplot package.
## Please report the issue at
## <https://github.com/GuangchuangYu/enrichplot/issues>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.

#display the first 3 enrichment plots
for (figure in head(all_figures, 3)) {
  print(figure)
}

```





0.18.5.2 Bar plots

- top pathways associated with Parental

```
topPathwaysDown <- fgseaRes[ES < 0][head(order(pval), n=20), pathway]
```

```
enrichment_results_topdown <- fgseaRes[which(fgseaRes$pathway %in% topPathwaysDown),]
```

```
#change the name to be just the first part of the name
```

```
enrichment_results_topdown$pathway <- unlist(lapply(enrichment_results_topdown$pathway, FUN=function(x){
```

```
#make sure that names are unique after shortening them (the same name can come from different datasets)
```

```
enrichment_results_topdown$pathway <- make.unique(enrichment_results_topdown$pathway)
```

```
##Calculate a score: the absolute value of the NES.
```

```
##(the pvalues of the top pathways are all very close to each other, so the NES is more informative)
```

```
enrichment_results_topdown$score = abs(enrichment_results_topdown$NES)
```

```
enrichmentTidy <- enrichment_results_topdown %>%
```

```
  as_tibble() %>%
```

```
  arrange(desc(score)) %>%
```

```
  slice_head(n = 20)
```

```
p = ggplot(enrichmentTidy, aes(reorder(pathway, score), score)) +
```

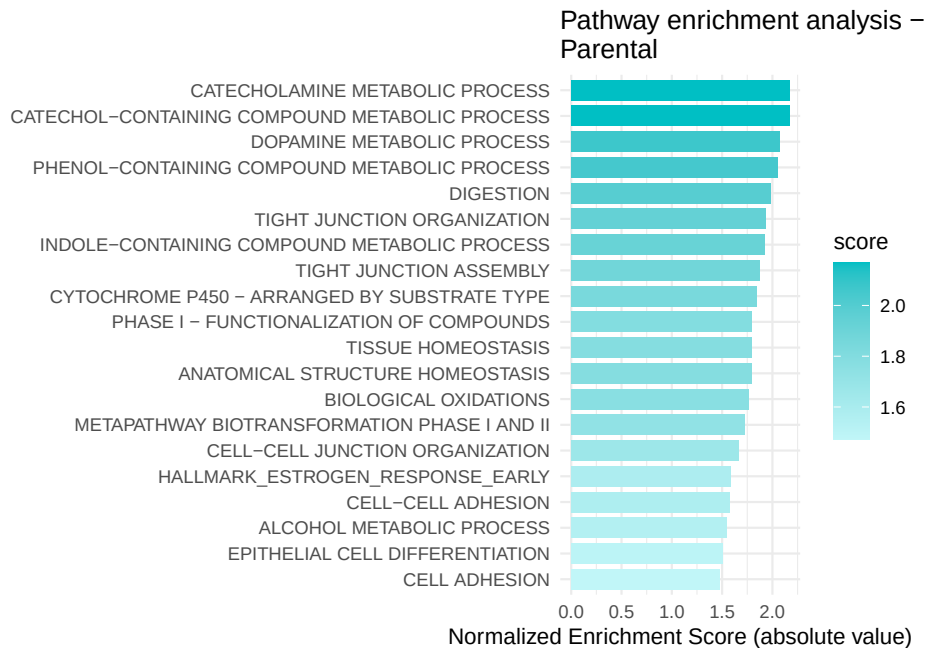
```
  geom_col(aes(fill = score), width=0.8) +
```

```
  scale_fill_gradient(low = "#C1F6F8", high = "#00BFC4") +
```

```
  coord_flip() +
```

```
labs(x="", y="Normalized Enrichment Score (absolute value)",
     title="Pathway enrichment analysis - \nParental") +
theme_minimal()
```

p



##YOUR TURN##

Generate the same bar plot for the top 20 Palbo-resistant results.
Change the colour of the bars to a different colour as
they are a different phenotype than the Parental set.

0.18.5.3 Dot plot

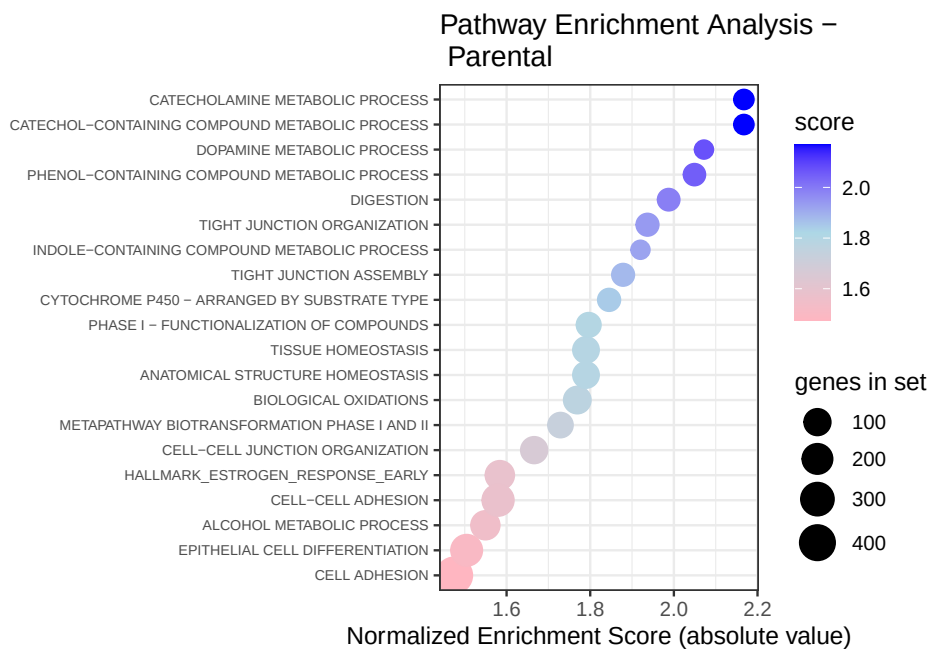
- top pathways associated with Parental

```
p = ggplot(enrichment_results_topdown, aes(
  y = fct_reorder(pathway, score),
  x = score,
  color = score,
  size = size
)) +
  geom_point() +
  scale_color_gradientn(
    colours = c("lightpink", "lightblue", "blue"),
```

```

    guide = guide_colorbar(reverse = FALSE, order = 1)
  ) +
  scale_size_continuous(range = c(4, 8), name = "genes in set") +
  theme_bw(base_size = 12) +
  xlab("Normalized Enrichment Score (absolute value)") +
  ylab(NULL) +
  ggtitle("Pathway Enrichment Analysis - \n Parental") +
  theme(
    axis.text.y = element_text(size = 7)
  )
p

```



##YOUR TURN##

Generate the same dot plot for the top 20 Palbo-resistant results.
Change the colour of the dots to a different colour range as
they are a different phenotype than the Parental set.

0.18.5.4 Ridge plots

- top pathways associated with either class

```

# Filter significant pathways (adjusted p-value < 0.05) and take
# the top 5 for each class (highest positive NES and most negative NES)
significant_pathways <- fgseaRes[padj < 0.05]

```

```

topPathways <- rbind(
  head(significant_pathways[NES > 0][order(-NES)], 5),
  head(significant_pathways[NES < 0][order(NES)], 5)
)

# Create a data.frame for plotting with the gene ranks for each of the top pathways
ridge_data <- rbindlist(lapply(topPathways$pathway, function(pw) {
  genes <- pathways[[pw]]
  data.frame(
    gene = genes,
    stat = ranks[genes],
    pathway = pw
  )
}))

# get NES for each of the top pathways
nes_map <- setNames(topPathways$NES, topPathways$pathway)

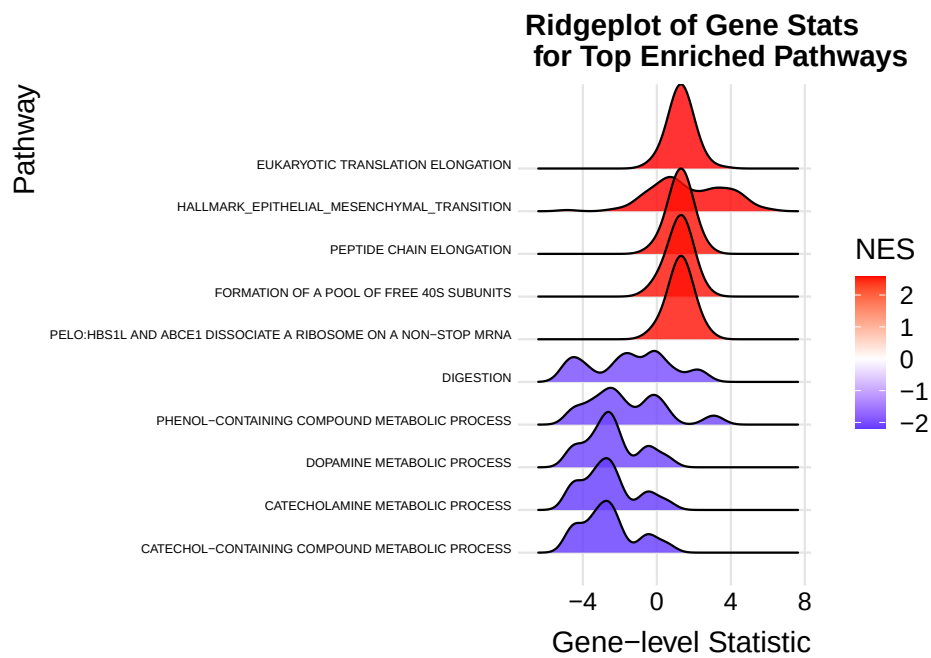
# Add NES values to ridge_data
ridge_data$NES <- nes_map[ridge_data$pathway]

#change the name to be just the first part of the name (because the actual names are too long)
#and make sure that names are unique after shortening them
short_names <- make.unique(unlist(lapply(topPathways$pathway, FUN=function(x){unlist(strsplit(x, "\\s"))}))
names(short_names) <- topPathways$pathway
ridge_data$pathway <- short_names[ridge_data$pathway]

#get rid of na values for genes found in the gene set but not ranked in our dataset.
ridge_data <- ridge_data[!is.na(ridge_data$stat),]

ggplot(ridge_data, aes(x = stat, y = reorder(pathway, NES), fill = NES)) +
  geom_density_ridges(scale = 2, alpha = 0.8) +
  scale_fill_gradient2(
    low = "blue", mid = "white", high = "red", midpoint = 0,
    name = "NES"
  ) +
  theme_ridges() +
  theme(legend.position = "right", axis.text.y = element_text(size = 6)) +
  labs(
    title = "Ridgeplot of Gene Stats\n for Top Enriched Pathways",
    x = "Gene-level Statistic",
    y = "Pathway"
  )

```



Dataset is coming from : <https://pubmed.ncbi.nlm.nih.gov/37384539/>

Pathway Enrichment Analysis reference paper: